Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

ЗВІТ

з лабораторної роботи №7
з дисципліни «Проектування програмних систем для мобільних пристроїв»

«Мова програмування PHP»

Варіант №4

Виконав:
студент 2 курсу, групи ІМ-51мн
Ковальов Олександр

Перевірив:
асистент, Нестерук А. О.

Дата здачі: 22.10.2026

КИЇВ – 2026

Мета. Вивчити основні принципи розробки додатків об'єктно-орієнтованою мовою програмування PHP та розглянути найбільш популярні фреймворки.

Завдання. Додати у програму, розроблену у лабораторній роботі №6, функціонал, який виконує визначення найближчого об'єкта, здатного забезпечити надання сервісу, необхідного програмі, відповідно до логіки її роботи (для програми «книги» найближчої книгарні, для застосунку «розклад потягів» найближчого залізничного вокзалу). Дані про доступні об'єкти підвантажувати з файлу. Запустити програму на емуляторі.

Завдання за варіантом. Доменна область: Абітурієнт.

Поля: ID, Прізвище, Ім'я, По-батькові, Адреса, Телефон, Оцінки.

Створити масив об'єктів.

Вивести:

а. список абітурієнтів, які мають незадовільні оцінки;

б. список абітурієнтів, середній бал у яких вище заданого;

в. вибрати задане число `N` абітурієнтів, що мають найбільш високій середній бал (вивести також повний список абітурієнтів, які мають напівпрохідний бал).

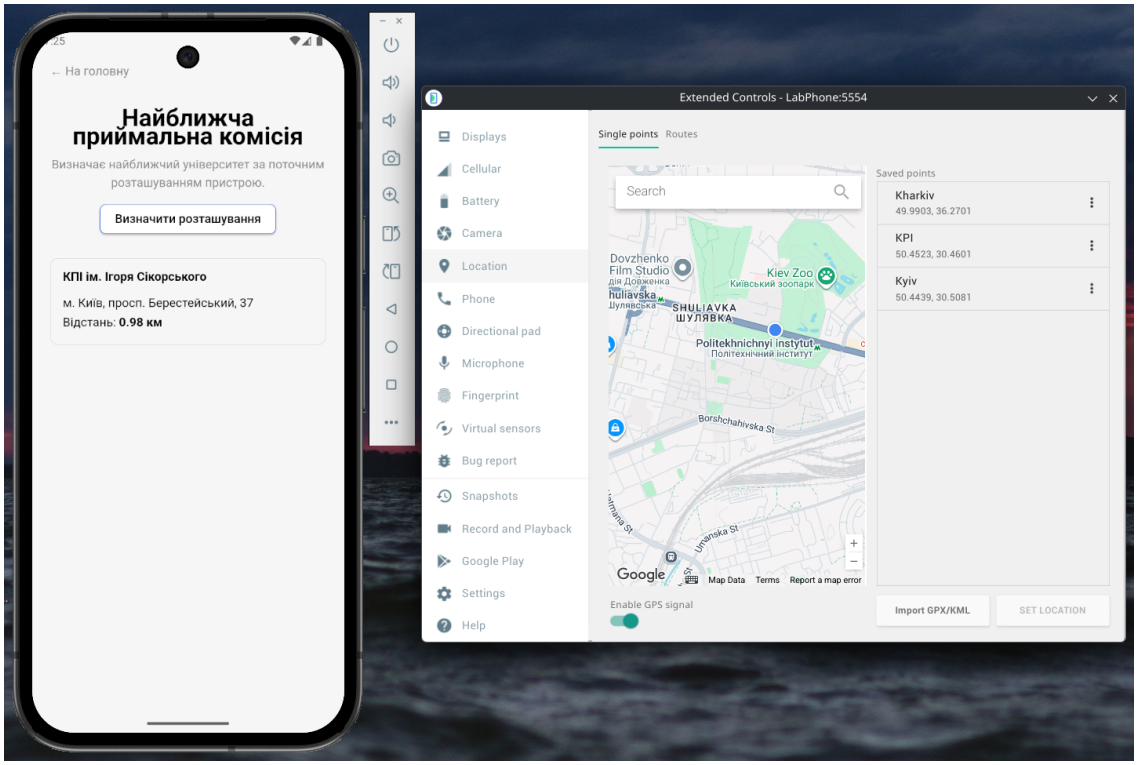
Хід роботи.

За основу взято застосунок із шостої лабораторної роботи без змін до наявної архітектури – додано лише новий функціонал, що визначає найближчий об'єкт, здатний надати сервіс, потрібний програмі, відповідно до логіки домену. Для домену «Абітурієнт» таким об'єктом природно виступає навчальний заклад: застосунок визначає, яка приймальна комісія (університет) знаходиться найближче до поточного розташування пристрою.

Перелік доступних закладів (назва, адреса, широта, довгота – 10 університетів у різних обласних центрах України) винесено у файл `resources/institutions.json`, який постачається разом із застосунком як ресурс Tauri (`bundle.resources` у `tauri.conf.json`) і підвантажується з файлу під час виконання команди, а не зашитий у код, як масив у `sample.rs`.

Обчислення відстані винесено в окремий чистий модуль `geo` – функцію `haversine_km()`, що обчислює відстань по великому колу між двома парами координат і нічого не знає ні про Tauri, ні про джерело цих координат. Команда `screen::nearest::find_nearest_university(lat, lon)` читає

`institutions.json`, обчислює відстань до кожного закладу через `haversine_km` і повертає найближчий разом із відстанню в кілометрах.



Під час реалізації виникли дві проблеми, специфічні саме для мобільної платформи.

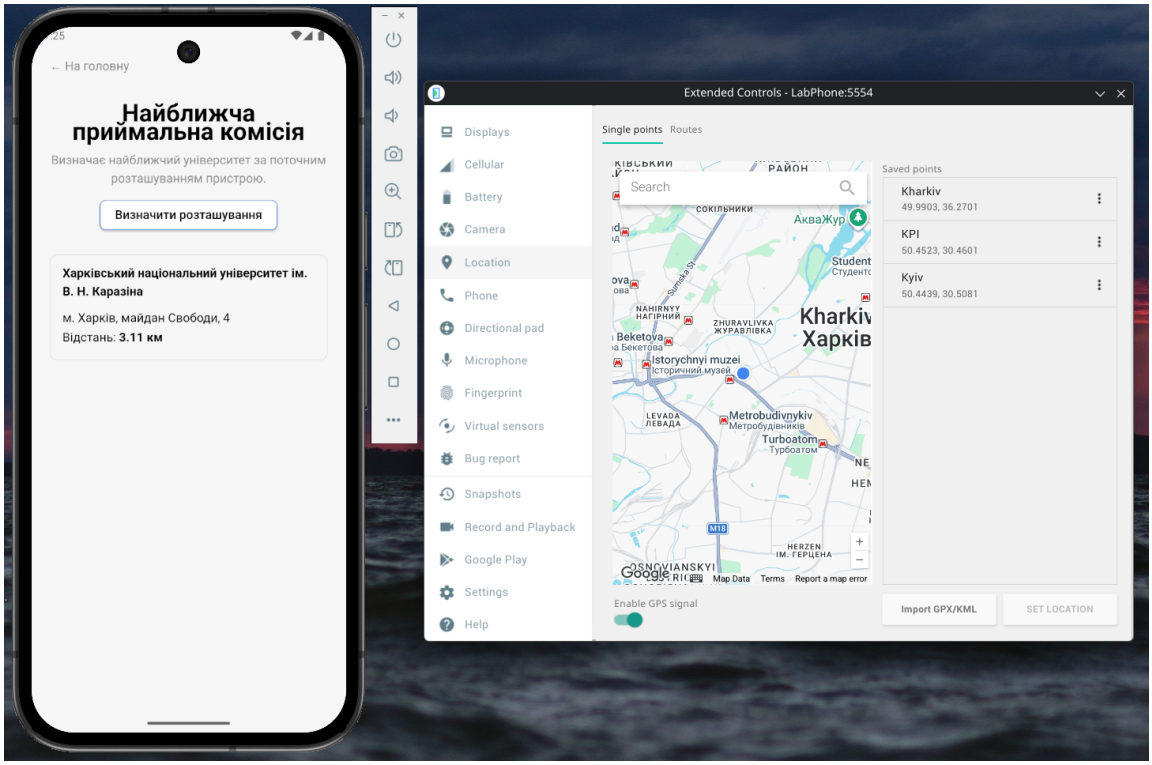
По-перше, читання ресурсного файлу через звичайний `std::fs::read_to_string` на Android завершувалось помилкою `No such file or directory`, оскільки на цій платформі bundled-ресурси лежать не на реальній файловій системі, а всередині APK-пакета, і шлях до них виглядає як `asset://localhost/...`. Вирішено підключенням `tauri-plugin-fs`: його метод `app.fs().read_to_string()` сам розпізнає такі шляхи й читає їх через Android `AssetManager`, а на інших платформах поводиться як звичайний виклик файлової системи.

По-друге, координати пристрою отримуються на боці фронтенду через офіційний плагін `tauri-plugin-geolocation` (команди `checkPermissions`/`requestPermissions`/`getCurrentPosition` із пакета `@tauri-apps/plugin-geolocation`) – це єдина частина функціоналу, що свідомо залишена на фронтенді, оскільки зчитування апаратного датчика є можливістю пристрою, а не бізнес-логікою; самі координати після отримання одразу передаються в Rust-команду, яка і виконує пошук найближчого закладу. Спершу виклик `getCurrentPosition()` без параметрів постійно завершувався помилкою `Location unavailable` після тайм-ауту: за замовчуванням запит іде з параметром `enableHighAccuracy: false`, що на Android відповідає пріоритету `PRIORITY_-`

`BALANCED_POWER_ACCURACY` й очікує радше мережевого визначення місця, якого в емуляторі не існує – є лише GPS-подібний фікс, заданий вручну через Extended Controls. Проблему усунуто, передавши `enableHighAccuracy: true` явно.

Оскільки `tauri-plugin-geolocation` підключений лише для мобільних платформ, його дозволи винесено в окремий файл `capabilities/mobile.json` із обмеженням `"platforms": ["iOS","android"]`: якби ці дозволи лишались у спільному `default.json`, перевірка дозволів під час збірки десктопної версії провалилася б, бо плагін там узагалі не є залежністю.

Коректність перевірено на емуляторі Android: координати пристрою поперемінно встановлювались через Extended Controls на Київ і на Харків, і в обох випадках застосунок коректно визначав найближчий заклад саме з відповідного міста – КПІ ім. Ігоря Сікорського для Києва та Харківський національний університет ім. В. Н. Каразіна для Харкова, що підтверджує правильність обчислення відстані й вибору мінімуму серед усіх закладів зі списку.



Висновок. У цій лабораторній роботі до застосунку «Абітурієнт» додано функціонал визначення найближчого об’єкта, що надає потрібний програмі сервіс.

Робота наочно показала дві практичні особливості розробки під Android: по-перше, файли, упаковані разом із застосунком, на Android доступні лише через системний `AssetManager`, а не напряму через файлову систему, тож їх читання потребує окремого плагіна; по-друге, точність запиту геолокації

(`enableHighAccuracy`) напряму впливає на те, яке джерело даних – мережеве чи супутникове – буде опитане, що особливо критично для емулятора, де реальних мережевих даних про розташування просто не існує.

Контрольні запитання.

1. **Назвіть основні особливості мови програмування PHP.** PHP є інтерпретованою скриптовою мовою загального призначення, яка спеціально розроблена для веб-розробки та генерації динамічних HTML-сторінок на стороні сервера. Її головні особливості включають динамічну типізацію, широку кросплатформність, вбудовану підтримку більшості сучасних систем керування базами даних, а також низький поріг входження завдяки простому синтаксису, що нагадує C та Perl.
2. **Назвіть переваги PHP-фреймворків.** Використання PHP-фреймворків значно пришвидшує процес розробки завдяки наявності готових модулів для маршрутизації, роботи з базами даних та аутентифікації користувачів. Вони також нав’язують розробникам чітку архітектуру, найчастіше патерн MVC, що сприяє написанню більш безпечного, тестованого та легкого для підтримки коду, водночас захищаючи від поширених вразливостей на зразок SQL-ін’єкцій.
3. **Охарактеризуйте фреймворк Laravel.** Це найпопулярніший сучасний PHP-фреймворк з відкритим вихідним кодом, який вирізняється елегантним синтаксисом та потужним набором інструментів. Він містить власну систему маршрутизації, ORM-систему Eloquent для зручної роботи з базами даних, механізм міграцій, шаблонізатор Blade та інтегровані інструменти для автоматизованого тестування.
4. **Назвіть основні етапи розробки та тестування проекту.** Життєвий цикл створення програмного забезпечення розпочинається зі збору вимог та проектування архітектури бази даних і програмних інтерфейсів. Наступними етапами є безпосереднє написання вихідного коду, проведення модульного та інтеграційного тестування для виявлення дефектів, після чого слідує розгортання готового продукту на робочому сервері з подальшим моніторингом.
5. **Опишіть використання Internet комунікацій у пристроях Android (HTTP).** Для взаємодії з веб-серверами через протокол HTTP в Android використовуються сучасні бібліотеки, такі як Retrofit або OkHttp, які забезпечують виконання асинхронних мережевих запитів. Цей підхід вимагає

обов'язкового оголошення дозволу на доступ до інтернету у файлі маніфесту та виконання всіх операцій обміну даними у фонових потоках.

6. **Опишіть використання Internet комунікацій у пристроях Android (FTP).** Обмін файлами через протокол FTP в Android-додатках реалізується за допомогою сторонніх Java-бібліотек, наприклад Apache Commons Net. Цей процес передбачає встановлення з'єднання з віддаленим сервером, проходження автентифікації, перехід у пасивний або активний режим передачі даних та виконання команд для завантаження або скачування бінарних файлів виключно у фоновому потоці.
7. **Опишіть використання Internet комунікацій у пристроях Android (e-mail).** Відправка електронних листів з Android-пристрою може здійснюватися двома основними шляхами. Перший метод полягає у використанні неявних намірів з дією `ACTION_SENDTO` для делегування завдання встановленому поштовому клієнту, тоді як другий варіант передбачає інтеграцію бібліотеки JavaMail API для прямого підключення до SMTP-сервера і непомітної відправки повідомлень безпосередньо з коду додатка.
8. **Опишіть використання Internet комунікацій у пристроях Android (сокети).** Робота з сокетами в Android базується на стандартних класах Java, таких як `Socket` та `ServerSocket`, які дозволяють встановлювати постійне двостороннє TCP або UDP з'єднання з іншими мережевими вузлами. Цей механізм є критично важливим для розробки додатків реального часу, оскільки він забезпечує мінімальну затримку при потоковій передачі байтових даних між клієнтом та сервером.
9. **Формати даних (html, xml, json).** HTML використовується переважно для візуальної розмітки та структурування веб-сторінок у браузері, тоді як XML є суворим ієрархічним форматом, що широко застосовується в Android для визначення макетів інтерфейсу та конфігураційних файлів. Натомість JSON, завдяки своїй легковагості та структурі пар ключ-значення, став фактичним стандартом для обміну інформацією між мобільними клієнтами та веб-сервісами.
10. **Опишіть відстеження розташування пристрою (GeoLocation).** Визначення географічних координат в Android здійснюється через системний сервіс `LocationManager` або більш сучасний Fused Location Provider API від Google Play Services. Ця технологія комбінує сигнали від супутників GPS, точок доступу Wi-Fi та базових станцій стільникової мережі для максимально швидкого та точного обчислення координат пристрою.

Лістинг.

geo.rs

```rust
//! Great-circle distance between two coordinates, used to rank candidate
//! institutions by how close they are to the device's current location.

const EARTH_RADIUS_KM: f64 = 6371.0;

#[allow(
clippy::suboptimal_flops,
reason = "Kept in standard textbook haversine form for readability"
)]
pub fn haversine_km(lat1: f64, lon1: f64, lat2: f64, lon2: f64) -> f64 {
    let (lat1, lon1, lat2, lon2) = (
    lat1.to_radians(),
    lon1.to_radians(),
    lat2.to_radians(),
    lon2.to_radians(),
    );

    let d_lat = lat2 - lat1;
    let d_lon = lon2 - lon1;

    let a = (d_lat / 2.0).sin().powi(2)
    + lat1.cos() * lat2.cos() * (d_lon / 2.0).sin().powi(2);
    let c = 2.0 * a.sqrt().asin();

    EARTH_RADIUS_KM * c
}

#[cfg(test)]
mod tests {
    use super::haversine_km;

    #[test]
    fn same_point_is_zero_distance() {
        assert_eq!(haversine_km(50.4547, 30.4467, 50.4547, 30.4467), 0.0);
    }

    #[test]
    fn kyiv_to_lviv_is_roughly_correct() {
        let distance = haversine_km(50.4547, 30.4467, 49.8389, 24.0243);
        assert!(
        (455.0..=470.0).contains(&distance),
        "expected ~455-470 km, got {distance}"
        );
    }
}
```